

Futaba S-BUS

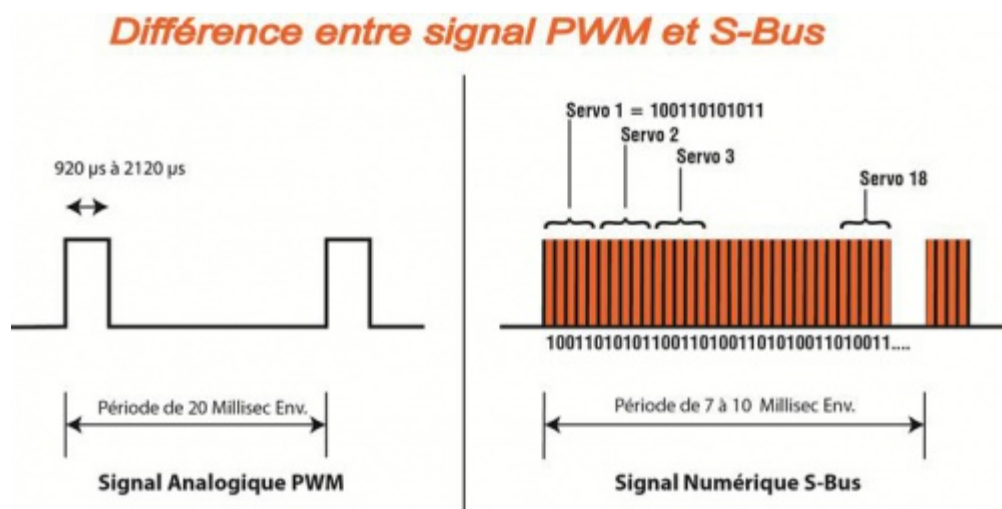
Introduction

The Futaba S-BUS protocol is a serial protocol to control servos. Up to 16 proportional and two digital channels are available. The protocol is derived from the very known RS232 protocol used everywhere. The signal must be first inverted. The frame is 8E2.

This board provide a complete electrical separation of RC gear and mbed controller. The S-BUS serial signal is converted and isolated by optocoupler. To keep control of the plane/car/ship an additional controller (ATTINY13) is on board. It monitors a standard servo signal (Master) and sends the serial S-BUS signal either to the mbed or directly to the servos. A special 'middle' position sends the S-BUS data to the servos and the mbed. In this mode the mbed can check stick positions and range without sending data to servos. Because the S-BUS data contains the Master signal, mbed knows about this mode.

S-BUS protocol

The protocol is 25 Byte long and is send every 14ms (analog mode) or 7ms (highspeed mode).



baudrate = 100,000 bit/s , One frame = 1 startbit + 8 databit + 1 paritybit + 2 stopbit (8E2)
The highest bit is send first. The logic is inverted (Level High = 1)

[startbyte] [data1] [data2] [data22] [flags][endbyte]

startbyte = 11110000b (0xF0)

data 1-22 = [ch1, 11bit][ch2, 11bit] [ch16, 11bit] (data ranges 0 - 2047)

channel 1 uses 8 bits from data1 and 3 bits from data2

channel 2 uses last 5 bits from data2 and 6 bits from data3

etc.

flags = bit7 = ch17 = digital channel (0x80)
bit6 = ch18 = digital channel (0x40)
bit5 = Frame lost, equivalent red LED on receiver (0x20)
bit4 = failsafe activated (0x10)
bit3 = n/a
bit2 = n/a
bit1 = n/a
bit0 = n/a

endbyte = 00000000b

SBUS to PPM decoder (Arduino)

/*

This example decodes a Futaba SBUS or compatible stream and convert it to PPM.

It is based on:

- <https://github.com/zendes/SBUS>
- <http://forum.arduino.cc/index.php?topic=99708.0>
- <https://mbed.org/users/Digixx/notebook/futaba-s-bus-controlled-by-mbed/>

There are some inexpensive SBUS to PPM decoders, such as

http://www.hobbyking.com/hobbyking/store/__37953__frsky_4_channel_s_bus_to_pwm_decoder.html

This code is more a proof of concept than a real alternative.

This code uses the RX0 as input. It's not possible to use a software serial port, because there are too many conflicting interrupts between serial input and PWM generation.

SoftwareServo could be replaced with the Servo library, but which cannot define the refresh delay. I used it just for fun.

Since RX0 is also used to communicate with the FTDI, the SBUS must be disconnected while flashing the Arduino.

Furthermore, the SBUS uses an inverted serial communication. Why should we do simple when it's so much fun to do it hard ? Thanks to Futaba for this oddity. The serial signal must be inverted, priorly to the RX0. This can be done with this inverter:

http://www.hobbyking.com/hobbyking/store/__24523__ZYX_S_S_BUS_Connection_Cable.html

(the shortest end goes to the receiver, the longest to the Arduino)

*/

```

#include <SoftwareServo.h>

// refresh delay for PPM (18 ms is good for any servo)
#define PPM_REFRESH_DELAY 18

// position goes from -100 to +100 - this ratio converts it to an
angle
// standard ratio is around 60
#define PPM_ANGLE          60

SoftwareServo myServo1;
SoftwareServo myServo2;
SoftwareServo myServo3;
SoftwareServo myServo4;

void setup () {
    Serial.begin (100000, 8E2);

    myServo1.attach (2);
    myServo2.attach (3);
    myServo3.attach (4);
    myServo4.attach (5);
}

void loop () {
    static byte          buffer[25];
    static int           channels[18];
    static int           errors = 0;
    static bool          failsafe = 0;
    static int           idx;
    static unsigned long last_refresh = 0;
    static int           lost = 0;

    byte b;
    int i;

    if (Serial.available ()) {
        b = Serial.read ();

        if (idx == 0 && b != 0x0F) { // start byte
            // error - wait for the start byte
        } else {
            buffer[idx++] = b; // fill the buffer
        }

        if (idx == 25) { // decode
            idx = 0;
            if (buffer[24] != 0x00) {
                errors++;
            } else {
                channels[0] = ((buffer[1]      | buffer[2] << 8) & 0x07FF);
                channels[1] = ((buffer[2] >> 3 | buffer[3] << 5) & 0x07FF);
                channels[2] = ((buffer[3] >> 6 | buffer[4] << 2 | buffer[5]
<< 10) & 0x07FF);
                channels[3] = ((buffer[5] >> 1 | buffer[6] << 7) & 0x07FF);
                channels[4] = ((buffer[6] >> 4 | buffer[7] << 4) & 0x07FF);
            }
        }
    }
}

```

```

    channels[5] = ((buffer[7] >> 7 | buffer[8] << 1 | buffer[9]
<< 9) & 0x07FF);
    channels[6] = ((buffer[9] >> 2 | buffer[10] << 6) &
0x07FF);
    channels[7] = ((buffer[10] >> 5 | buffer[11] << 3) &
0x07FF);
    channels[8] = ((buffer[12] | buffer[13] << 8) & 0x07FF);
    channels[9] = ((buffer[13] >> 3 | buffer[14] << 5) &
0x07FF);
    channels[10] = ((buffer[14] >> 6 | buffer[15] << 2 |
buffer[16] << 10) & 0x07FF);
    channels[11] = ((buffer[16] >> 1 | buffer[17] << 7) &
0x07FF);
    channels[12] = ((buffer[17] >> 4 | buffer[18] << 4) &
0x07FF);
    channels[13] = ((buffer[18] >> 7 | buffer[19] << 1 |
buffer[20] << 9) & 0x07FF);
    channels[14] = ((buffer[20] >> 2 | buffer[21] << 6) &
0x07FF);
    channels[15] = ((buffer[21] >> 5 | buffer[22] << 3) &
0x07FF);
    channels[16] = ((buffer[23]) & 0x0001) ? 2047 : 0;
    channels[17] = ((buffer[23] >> 1) & 0x0001) ? 2047 : 0;

    failsafe = ((buffer[23] >> 3) & 0x0001) ? 1 : 0;
    if ((buffer[23] >> 2) & 0x0001) lost++;

    if ((millis () - last_refresh) >= PPM_REFRESH_DELAY) {
        myServo1.write (toPct (channels[0]) * PPM_ANGLE / 100 +
90);
        myServo2.write (toPct (channels[1]) * PPM_ANGLE / 100 +
90);
        myServo3.write (toPct (channels[2]) * PPM_ANGLE / 100 +
90);
        myServo4.write (toPct (channels[3]) * PPM_ANGLE / 100 +
90);

        SoftwareServo::refresh ();
        last_refresh = millis ();
    }
}
}
}
}

int toPct (int val) {
    return (int) lround (val / 9.92) - 100;
}

```

